**Aubrée Romain** **Beauvallet**
**13/11/2026** **Semestre 3**
**SLAM**

**Groupe 1**

# Bloc 2 Projet Colloc’

**Contexte** **2**
**Partie 1 : Conception et base de données** **3**
**Partie 2 : Comptes et colocations** **5**
**Partie 3 : La sécurité par RLS** **6**
**Partie 4 : L’architecture Flutter** **7**
**Partie 5 : Dépenses, soldes et remboursements** **8**
**Partie 6 : Le temps réel (tâches, courses, chat)** **10**
**Partie 7 : Robustesse et qualité** **13**
**Partie 8 : Bonus — les statistiques** **16**
**Conclusion** **17**

# Contexte

Pour ce projet, on devait réaliser une **application mobile de gestion de colocation** avec **Flutter** et **Supabase**. Le principe : des colocataires partagent le même logement et ont besoin de suivre leurs **dépenses communes**, de se répartir les **tâches ménagères**, de gérer une **liste de courses** et de discuter dans un **chat**.

On a travaillé à deux : **Louis HERZOG** et **Romain AUBREE**. Contrairement aux TP précédents, **aucun code n'était fourni** : seulement un cahier des charges. On est partis d'un **flutter create** vide et du package **supabase_flutter**, et on a tout construit nous-mêmes.

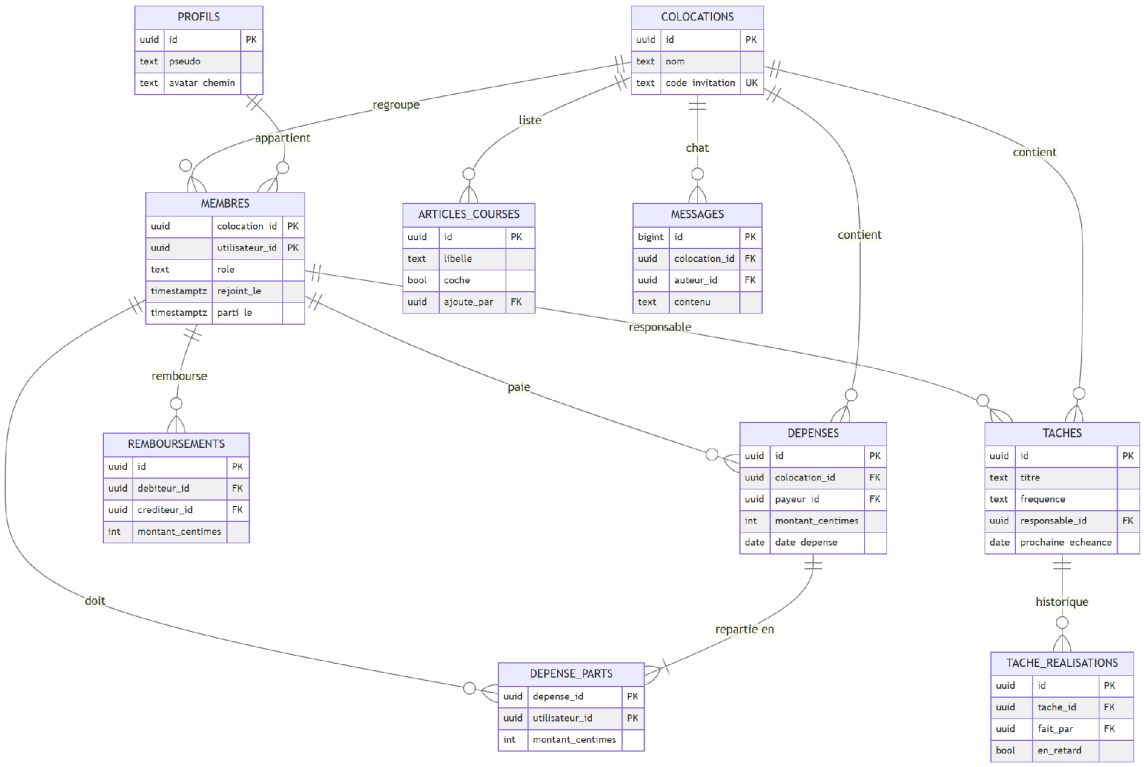
Le sujet impose aussi des **règles strictes** : la sécurité repose **uniquement sur la RLS**, la clé secrète **service_role** ne doit apparaître nulle part, chacun fait ses propres **commits**, et on tient un **journal de bord** et un fichier de **décisions d'architecture**. Le projet est découpé en **5 jalons**, de la conception jusqu'à la soutenance.

Le découpage en **jalons** suit le sujet : (1) la **conception** sans code, (2) les **fondations** (comptes et colocs, avec une démonstration RLS en direct), (3) les **dépenses et les soldes**, (4) le **temps réel**, et (5) les **finitions, les tests et la soutenance**.

Le projet est disponible sur GitHub (***https://github.com/ShadowGcraft/Colloc-gestion***).

# Partie 1 : Conception et base de données

Avant d'écrire la moindre ligne de Flutter, on a conçu la **base de données** (jalon 1). Voici le **schéma** que l'on a dessiné :




Il comprend les tables suivantes :

- **profils** : un profil par utilisateur (pseudo, avatar).
- **colocations** : les colocs, avec un **code d'invitation** unique.
- **membres** : qui appartient à quelle coloc, avec son **rôle** (admin ou membre).
- **depenses** et **depense_parts** : une dépense et sa répartition entre les participants.
- **remboursements** : les remboursements entre colocataires.
- **taches** et **tache_realisations** : les tâches et leur historique.
- **articles_courses** et **messages** : la liste de courses et le chat.

On a dû trancher plusieurs **questions de conception** (justifiées dans DECISIONS.md) :

**Les montants.** On stocke tout en **centimes entiers**, jamais en nombre à virgule. Les **float** font des erreurs d'arrondi (0,1 + 0,2 ≠ 0,3) ; avec des entiers, les calculs sont exacts et la **somme des soldes vaut exactement 0**.

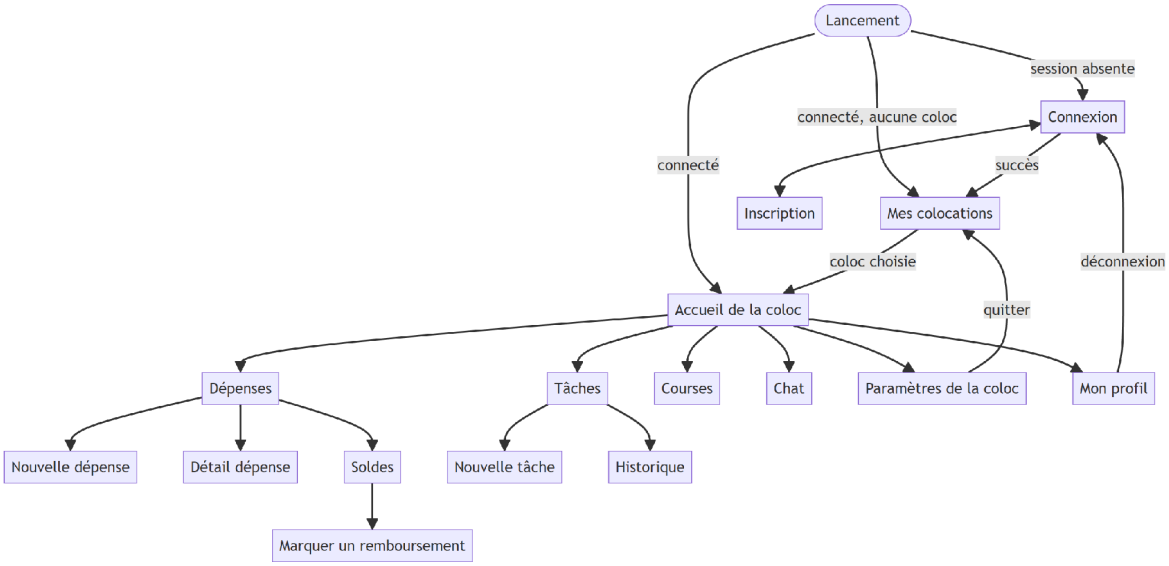
**L'appartenance.** « Un utilisateur appartient à une coloc avec un rôle » est une relation **plusieurs-à-plusieurs qui porte un attribut**, donc une **table membres**. Un utilisateur peut ainsi être dans plusieurs colocs à la fois.

**La dépense.** Une seule table ne suffit pas : on a **depenses** (montant, payeur, date…) et **depense_parts** (une ligne par participant). Les parts sont **toujours stockées**, même en parts égales.

On a aussi écrit la **matrice d'autorisations** : qui peut lire (L), créer (C), modifier (M) ou supprimer (S) quoi.

| Donnée | Non-membre | Membre | Auteur | Admin |
|---|---|---|---|---|
| **colocations** | Créer (RPC) | Lire | — | M · S · régénérer le code |
| **membres** | Rejoindre (RPC) | Lire · quitter | — | Retirer un membre (RPC) |
| **depenses** | — | Lire · Créer (RPC) | M · S | M · S |
| **remboursements** | — | Lire · Créer | Supprimer | Supprimer |
| **taches** | — | Lire · Créer · faire | M · S | M · S |
| **articles_courses** | — | Lire · Créer · cocher · S | — | — |
| **messages** | — | Lire · Créer | Supprimer | Supprimer |
| **vue soldes** | — | Lire | — | — |

Enfin, on a maquetté les **5 écrans clés sur papier** et dessiné le **graphe de navigation** :

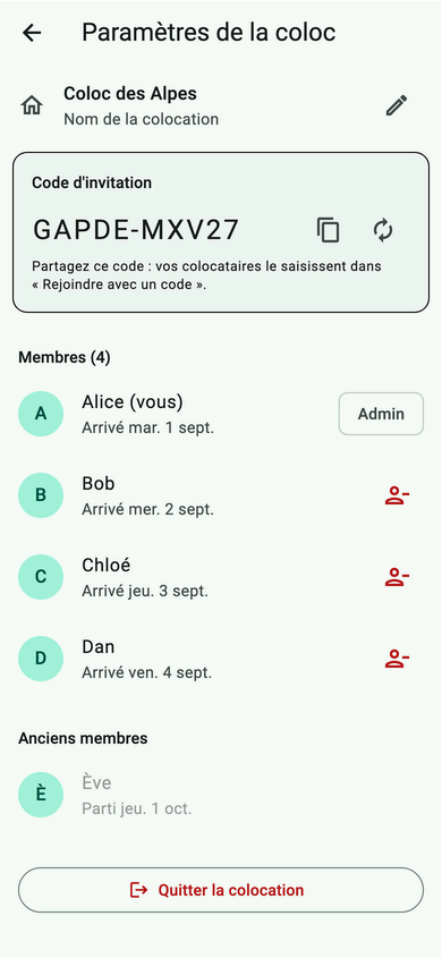



# Partie 2 : Comptes et colocations

La première brique de l'appli, c'est la gestion des **comptes** et des **colocations**. On a implémenté :

- **Inscription**, **connexion** et **déconnexion** par e-mail et mot de passe.
- **Profil** : modifier son **pseudo** et son **avatar** (photo stockée dans Storage).
- **Créer** une colocation, ou la **rejoindre avec un code d'invitation**.
- **Appartenir à plusieurs colocations** et **basculer** de l'une à l'autre.
- **Rôles** : **admin** (le créateur) et **membre**.
- **Seul l'admin** peut **retirer un membre** ou **régénérer le code d'invitation**.

La redirection est **automatique** : l'appli écoute l'état de la session et affiche l'écran de connexion, la liste des colocs ou l'accueil selon le cas. L'écran des **paramètres de la coloc** regroupe les membres, le code et le bouton « quitter ».




# Partie 3 : La sécurité par RLS

La règle d'or du sujet : **aucune confiance accordée au client**. Toute la sécurité est en base, avec la **RLS (Row Level Security)**. Même avec la clé publique et un vrai compte, on ne peut pas voir les données d'une coloc dont on n'est pas membre. On a rencontré plusieurs pièges, qui sont les **points techniques** du sujet :

**Récursion des policies.** Une policy de **membres** qui relit **membres** provoque une **récursion infinie**. On la contourne avec des fonctions **SECURITY DEFINER** dans un schéma privé, qui lisent **membres** sans repasser par la RLS.

**Rejoindre une coloc.** Un non-membre ne peut rien lire : il passe par la fonction **rejoindre_colocation(code)**. Le code fait 10 caractères tirés au hasard, impossible à deviner.

**Intégrité.** Une dépense ne peut pas référencer un payeur non-membre : une **clé étrangère composite** l'interdit. C'est une **contrainte**, donc incontournable.

**Départ d'un membre.** On ne supprime personne : on renseigne **parti_le**. Ses dépenses restent valides, il apparaît encore dans les soldes, mais il perd l'accès immédiatement.

Enfin, pour l'**audit de sécurité croisé**, un compte isolé (Ève, seule dans sa coloc) sert de cible : une requête **curl** directe vers une autre coloc est bien **refusée ou vide**.

# Partie 4 : L'architecture Flutter

Côté application, on a séparé proprement le code. Les dossiers sont : **modeles** (classes Dart), **donnees** (les dépôts), **etat** (session, réseau), **vues** (écrans) et **widgets** / **outils** réutilisables.

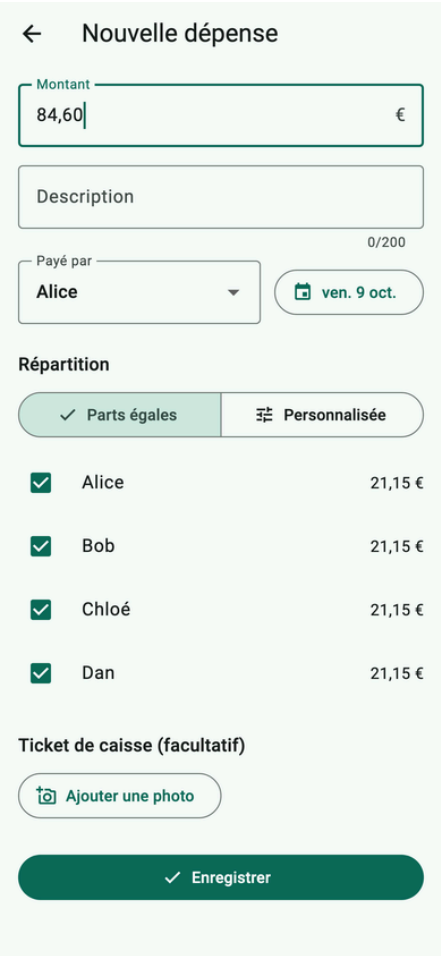
Le point important : **aucun écran n'appelle Supabase directement**. Tout passe par une **couche de dépôts**. Les écrans ne connaissent que des interfaces, ce qui permet de les **tester** en remplaçant les dépôts par de faux objets en mémoire.

Pour l'**état**, on a suivi la méthode officielle de Flutter : un **ChangeNotifier** par écran et une **Session** partagée. On n'a ajouté **aucun package superflu** : seulement **supabase_flutter** et **image_picker**.

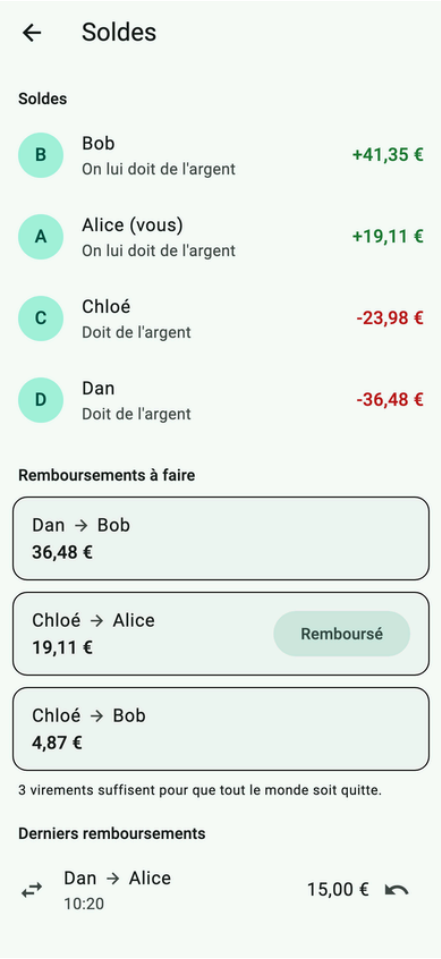
# Partie 5 : Dépenses, soldes et remboursements

C’est le cœur du projet (jalon 3). Les fonctions :

- **Ajouter une dépense** : montant, payeur, description, date.
- **Répartition** entre certains membres : **parts égales** ou **personnalisées**.
- **Photo du ticket** de caisse (stockée dans Storage).
- **Liste** des dépenses paginée et **écran de détail**.
- **Écran « Soldes »** : qui doit combien à qui, avec le **minimum de remboursements**.
- **Marquer un remboursement comme effectué**.



Le calcul des **soldes** se fait en **SQL**, avec une **vue** qui renvoie une ligne par membre (ce qu'il a payé moins ce qu'il doit). Avec 2 000 dépenses, la base renvoie quelques lignes au lieu de tout recalculer côté téléphone.




**Les arrondis.** 10 € entre 3 personnes font 3,34 + 3,33 + 3,33 : le centime en trop va au premier participant, et un **contrôle en base** refuse toute dépense dont la somme des parts n'égale pas le total.

**L'atomicité.** Créer une dépense et ses parts réussit ou échoue **en bloc**, grâce à une fonction **creer_depense** exécutée dans une seule transaction.

**La simplification des dettes.** On calcule le **minimum de virements** avec un algorithme **exact** (pas glouton), couvert par des tests unitaires. Pour le jalon 3, on a **prédit les soldes sur papier** puis vérifié que l'appli donne le même résultat, de **somme exactement nulle**.

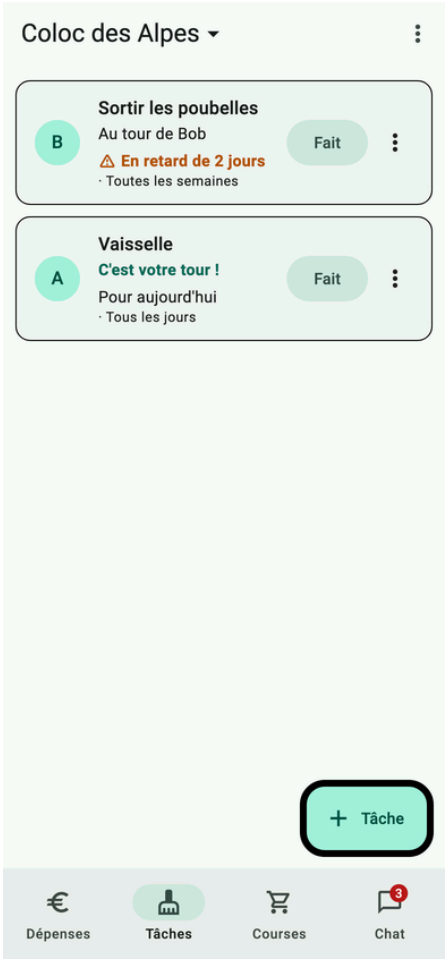
Voici les soldes prévus pour ce jeu de test (montants en centimes) :

| Membre | A payé | Parts dues | Solde |
|---|---|---|---|
| **Alice** | 10 010 | 8 099 | +1 911 |
| **Bob** | 12 000 | 7 865 | +4 135 |
| **Chloé** | 3 700 | 6 098 | −2 398 |
| **Dan** | 2 999 | 6 647 | −3 648 |
| **Somme** | — | — | 0 |

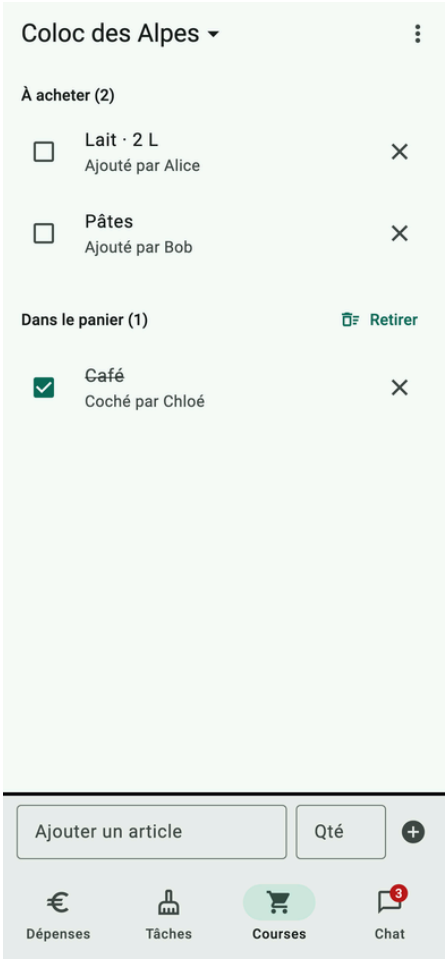
# Partie 6 : Le temps réel (tâches, courses, chat)

Trois écrans fonctionnent en **temps réel** grâce à Supabase Realtime (jalon 4).

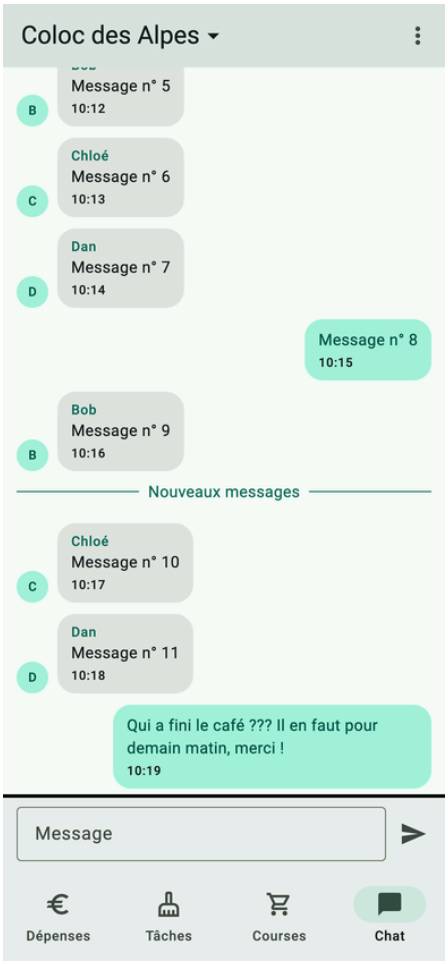
**Les tâches.** Elles sont **récurrentes** (quotidiennes ou hebdomadaires) avec **rotation automatique** entre les membres. On peut les **marquer faites**, consulter l'**historique** et voir un **indicateur de retard**. La rotation est gérée **en base**, dans une seule transaction.



**Les courses.** La liste partagée se met à jour en direct : on peut **ajouter**, **cocher** et **supprimer** un article, et on voit **qui a ajouté ou coché quoi**.



**Le chat.** Les messages arrivent en temps réel, l'historique se charge **progressivement** en remontant, et un **indicateur de nouveaux messages** prévient quand on n'est pas en bas. Ça reste rapide même avec **10 000 messages**.

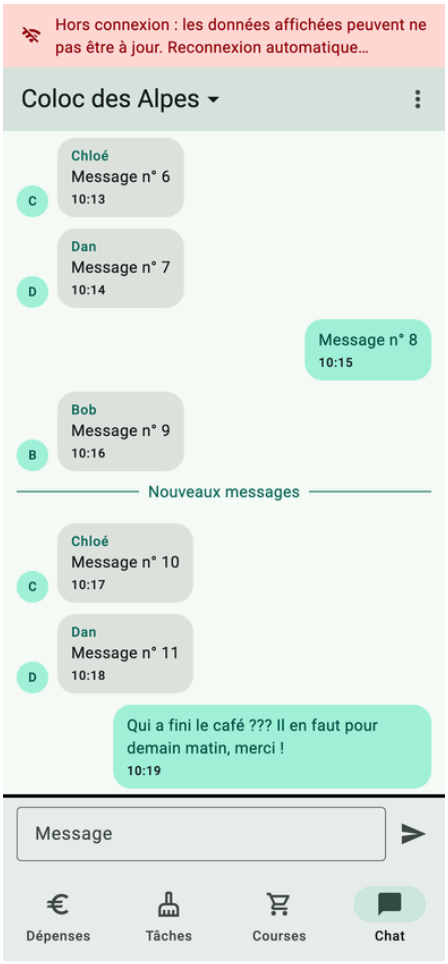



**Gestion des abonnements.** À chaque écran, on **ouvre** le canal à l'entrée, on le **ferme** à la sortie, et on **recharge tout à la reconnexion**. Le filtre par coloc est côté serveur. Le jalon 4 le vérifie : on coupe le réseau sur un appareil et tout se resynchronise au retour.

## Partie 7 : Robustesse et qualité

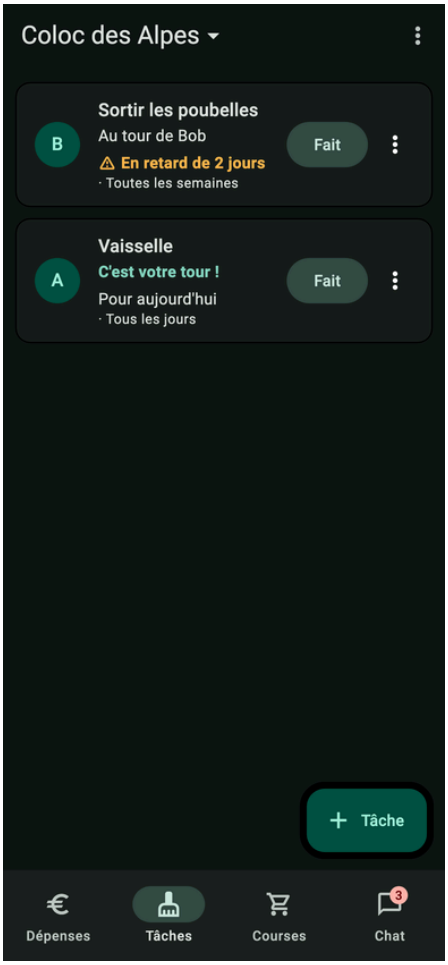
On a soigné les cas qui cassent souvent une appli :

- **États** de **chargement**, d'**erreur** et **liste vide** sur tous les écrans.
- **Mode hors connexion** : un **bandeau** prévient, l'appli sonde le serveur, et au retour chaque écran se recharge seul.
- **Thème clair et thème sombre** sur toute l'interface.



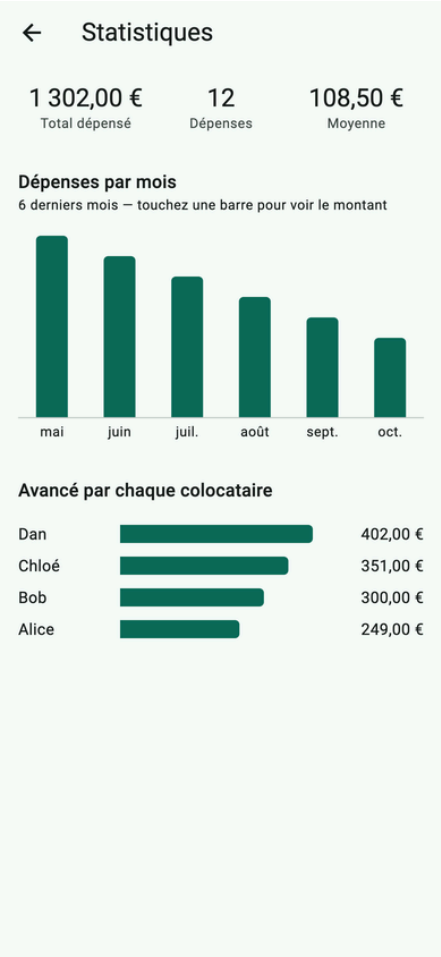

Côté **tests**, on a quatre niveaux : **unitaires** (soldes, simplification), **de widgets** (formulaires et tous les écrans en clair et sombre), **SQL** (pgTAP : soldes et isolement RLS) et **d'intégration** contre un vrai Supabase local, plus un scénario **de bout en bout** qui rejoue le jalon 4.

Résultat : **22 tests Flutter** et **11 tests SQL** au vert, et **flutter analyze** ne remonte aucun problème.



# Partie 8 : Bonus — les statistiques

Comme **fonctionnalité bonus**, on a ajouté un écran **Statistiques** : le total des dépenses par mois et la part de chaque colocataire, dessinés avec de simples widgets, **sans aucun package supplémentaire**. Cela réutilise les données déjà présentes.



# Conclusion

Au final, toutes les **fonctionnalités obligatoires** marchent : comptes et colocations, dépenses et soldes exacts, tâches avec rotation, courses et chat en temps réel. La **sécurité repose uniquement sur la RLS**, la clé secrète n'apparaît nulle part, et l'audit croisé ne révèle aucune fuite.

Côté démarche, on a gardé un **dépôt Git** avec des commits par personne, un **JOURNAL.md** et un **DECISIONS.md** qui justifie chaque choix. Ce projet nous a surtout appris à **penser la base de données et la sécurité avant le code**, et à faire confiance à la base plutôt qu'au client.

Enfin, on a versionné le projet avec **Git** (des commits réguliers, par personne et par fonctionnalité) et poussé le dépôt sur **GitHub**.